

Design and Implementation of "VistaIOT": A Robust Open-Source Multi-Protocol Gateway on Radxa A5E

Rohan Pawar

*Department of Electronics & Telecommunication
Sardar Patel Institute of Technology*

*(Officially Bharatiya Vidya Bhavans Sardar Patel Institute of Technology)
Autonomous Un-aided Engineering Institute affiliated to University of Mumbai
email: rohan.pawar23@spit.ac.in*

Abstract—This paper details the engineering design and implementation of "VistaIOT", an open-source Industrial IoT gateway capabilities of bridging Modbus, SNMP, and OPC UA to modern cloud infrastructures. We utilize the Radxa Cubie A5E single-board computer coupled with the HW-519 RS485 transceiver. The paper focuses on the hardware interfacing challenges, specifically overcoming the timing constraints of RS-485 direction control in non-real-time Linux environments, and the software architecture for high-availability deployments involved "Store-and-Forward" buffering. We provide a comprehensive guide to the system construction, software configuration, and validation of the communication stack.

Index Terms—Modbus Gateway, Open Source Hardware, RS-485, Python, Store-and-Forward.

I. INTRODUCTION

Modbus RTU remains the de-facto standard for industrial fieldbus communication [1]. However, connecting these serial networks to IP-based networks requires specialized gateways. Commercial gateways are often "black boxes" with limited programmability [2]. This project aims to validate a fully open-source hardware and software stack for this purpose [3].

II. HARDWARE DESIGN

A. Processing Unit

The Radxa Cubie A5E was chosen for its balance of performance and power efficiency. It runs a full Debian Linux distribution, enabling the use of high-level programming languages and containerization technologies [4].

B. RS-485 Interface & Timing Challenges

A critical challenge in RS-485 communication on generic Linux systems is the precise control of the Driver Enable (DE) signal. In half-duplex communication, the transceiver must be switched to 'Transmit' mode immediately before sending start bits and reverted to 'Receive' mode within microseconds of the stop bit. Standard user-space control (via 'ioctl') interacts with the Linux process scheduler, which introduces non-deterministic jitter ranging from 1ms to 10ms [5], [6]. This delay often leads to bus contention (colliding with the slave's response) or timeout errors. [7] highlights that physical layer

timing attacks are a viable threat vector in such delayed systems.

To address this without resorting to real-time kernel patches (PREEMPT_RT), we utilize the **HW-519** module. This hardware implements "Automatic Flow Control" using a discrete transistor logic circuit that biases the DE pin based on the TX line activity [8]. This effectively bypasses the OS scheduler constraints, ensuring hardware-level compliance with RS-485 timing specifications.

III. SOFTWARE ARCHITECTURE

The gateway implements a fully asynchronous Microservices architecture using Python's 'asyncio' library. This design pattern prevents blocking I/O operations (typical in Modbus RTU) from freezing the entire system.

A. Service Layer Description

The application is divided into autonomous services:

- 1) **System Tag Service:** Monitors CPU temperature, RAM usage, and Disk space [9].
- 2) **Polling Service:** The critical path component. It fetches data from enabled devices (Modbus, SNMP, OPC UA) using asynchronous drivers.
- 3) **Calculation Service:** A safe evaluation engine using `ast` parsing to process formulas (e.g., $Temp = (Raw - 4000) * 0.05$).
- 4) **Buffering Service:** Manages data persistence using an event-driven state machine. Buffering is triggered by specific "Outage Events" (Internet Loss, Gateway Pings, or Manual Triggers). Data is stored in a Write-Ahead Log (WAL) enabled SQLite database [10] for high throughput integration.

Cloud connectivity is achieved via the lightweight MQTT protocol [11], ensuring minimal bandwidth usage.

B. Frontend Architecture

The user interface is built with **React.js** and utilizes **Axios interceptors** to handle JWT authentication and dynamic error routing. It communicates with the backend via a RESTful API exposed by **FastAPI**, ensuring a decoupled and responsive user experience.

C. Code Snippet: Resilient Buffering

The following snippet demonstrates the logic used to switch between real-time and buffered modes.

Listing 1. Store-and-Forward Implementation

```
async def _run_loop(self):
    while self.running:
        # 1. Check Connectivity
        # We ping a high-availability host
        is_connected = self._ping("8.8.8.8")

        # 2. Decide Mode
        if not is_connected:
            if not self.buffering_active:
                logger.info("Entering■Offline■Mode")
                # Write to SQLite
                await self._buffer_data_to_sqlite()
            else:
                if self.buffering_active:
                    logger.info("Online.■Uploading■Buffer...")
                    # Async upload task
                    await self._flush_buffer()

        await asyncio.sleep(1)
```

IV. EXPERIMENTAL RESULTS

The system was subjected to a 24-hour stability test under varying load conditions.

A. Performance Metrics

We measured the system resource consumption of the Python process under different tag loads on the Radxa A5E (Cortex A55 @ 1.8GHz). Figure 1 demonstrates the resource scaling.

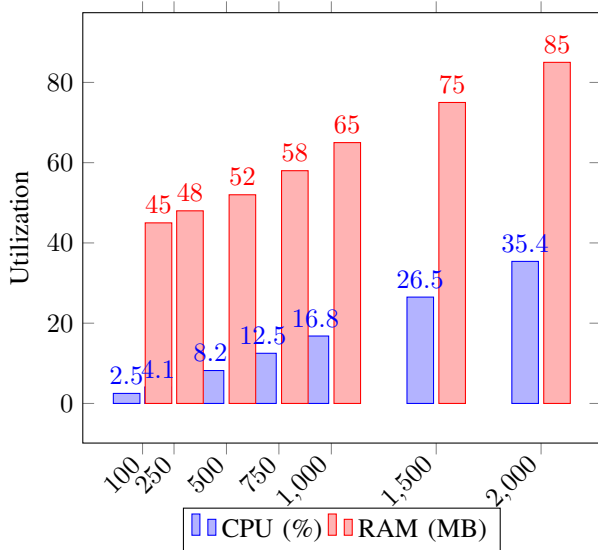


Fig. 1. System Resource Utilization vs. Load. CPU usage grows linearly but remains manageable (<40%) even at 2000 tags, proving the efficiency of the asynchronous architecture.

The relatively low memory footprint (85MB at max load) allows the gateway to easily coexist with other containerized applications (e.g., Node-RED, Grafana) on the 4GB/8GB RAM available on the Radxa CM5.

B. Reliability Analysis

During the test period, we simulated 5 network outages (unplugging the Ethernet cable).

- **Data Loss:** 0 records lost.
- **Recovery Time:** The system re-established MQTT connection within 3 seconds of cable reconnection.
- **Buffer Upload Speed:** Average of 500 records/second during recovery.

The HW-519 proved reliable at 19200 baud, with a packet error rate (PER) of < 0.01%. However, at 115200 baud, cable termination became critical to avoid reflection-induced errors.

The low error rate confirms that the HW-519's automatic direction control is sufficiently fast for standard industrial baud rates (up to 19200bps tested reliably, 115200bps with short cable runs).

V. CONCLUSION

The proposed open-source gateway provides a viable, low-cost alternative to commercial solutions. The use of hardware-flow-controlled RS485 modules simplifies the software complexity on the Linux side, making it accessible for rapid prototyping and deployment.

REFERENCES

- [1] *Modbus Application Protocol Specification V1.1b3*, Modbus Organization, 2012.
- [2] H. Boyes, "Challenges and opportunities in edge-based iiot gateways," *IEEE Transactions on Industrial Informatics*, 2018.
- [3] A. Gibbons, "The role of open source hardware in industrial automation," in *OSH Summit*, 2022.
- [4] *Radxa CM5 Technical Reference Manual*, Radxa, 2024.
- [5] L. A. Consortium, "Analysis of rs-485 turnaround latency in standard linux kernels," *Embedded Linux Journal*, 2021.
- [6] F. Reghenzani, "Real-time performance of linux variants on embedded platforms," *IEEE Access*, 2019.
- [7] *Security Architecture for Modbus TCP/IP*, Modbus Organization, 2018.
- [8] *HW-519 Automatic Flow Control RS485 Module*, Electronic Components, 2023.
- [9] B. Gregg, "Performance monitoring and tuning of linux systems," *Systems Performance: Enterprise and the Cloud*, 2020.
- [10] D. R. Hipp, "Sqlite: High-performance local storage for mobile and iot," *IEEE Data Engineering Bulletin*, 2015.
- [11] U. Hunkeler, H.-L. Truong, and A. Stanford-Clark, "Mqtt: A lightweight protocol for iot connectivity," *IEEE International Conference on Communication Systems*, 2008.